

SCAFFOLDS | SEARCH | META-UTILITY | GOVERNANCE

WHEN THE IMPROVER IMPROVES ITSELF

A pedagogical introduction to the Self-Taught Optimizer (STOP)

How a fixed language model can become more effective by improving the scaffold that organizes generation, evaluation, search, and selection.

TEACHING NOTE OVERVIEW

The paper *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation* asks a deceptively simple question: if we can build a program that uses a language model to improve other programs, can that program also improve itself? The answer explored by STOP is not a story about an artificial intelligence rewriting its own brain. The underlying language model remains fixed. What changes is the *scaffold*: the procedure that decides how to use the model, how many alternatives to generate, how to evaluate them, how to search, and how to select better solutions. This teaching note explains the essence of the STOP architecture in non-technical terms. The central lesson is that the object of optimization can move from the *answer*, to the *search process*, and finally to the *optimizer itself*. This shift provides a useful conceptual bridge from ordinary prompting toward self-improving agentic systems.

Alejandro Reynoso

AI Systems / Optimization / Governance

September 2026

Pedagogical Teaching Note

Primary source: Zelikman, Lorch, Mackey, and Kalai, *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation*, COLM 2024.

The Core Question

The central question behind STOP is:

If a system can improve a solution, can it also improve the process that performs the improvement?

At first sight this may sound like a small variation on ordinary use of a language model. It is not. It changes the object of attention.

The simplest use of a language model looks like this:

Human request → Language model → Answer

For example, a user may ask a model to write a Python program. The model produces one program, and the interaction ends.

A slightly more sophisticated process would ask the model to review its answer and produce a better version:

Generate → Review → Improve

A still better process might ask for several alternatives, test them, and keep the best:

Generate candidates → Evaluate → Select best

At that point, performance depends on more than the language model itself. It also depends on the *system around the model*: how many candidates are generated, how they are tested, how much exploration is allowed, how the best alternatives are preserved, and when the process stops.

STOP calls this surrounding structure a *scaffold*. The research begins from the observation that scaffolding can materially improve what a fixed language model can accomplish. The conceptual leap is then straightforward but profound:

Designing a good scaffold is itself an optimization problem.

Once that is recognized, the system can try to improve not only the answer, but also the machinery used to find the answer [1].

The Seed Improver

STOP begins with a very simple program called an *improver*.

Suppose we already have a program that solves some problem, but not very well. Call it the *initial solution*. We also have a way to measure how good a proposed replacement is. The paper calls this measure a *utility function*.

Conceptually:

Candidate solution → Evaluation → Score

The seed improver follows a simple recipe:

1. Give the current solution to the language model.
2. Ask the model to propose several improved versions.
3. Evaluate the proposed versions.
4. Compare their scores.
5. Return the best candidate.

The important feature is the separation between *generation* and *evaluation*. The language model proposes ideas, but it does not simply declare that its newest answer is better. An external scoring procedure decides which candidate performs best.

This is already more powerful than ordinary prompting because the system is allowed to search.

A useful analogy is a research laboratory. A scientist does not normally accept the first hypothesis that comes to mind. Several hypotheses may be proposed, experiments are performed, and evidence determines which ideas survive. The improver plays a similar role: it converts the language model from a one-shot answer generator into a component inside a search-and-selection process.

The Crucial Move: Improve the Improver

The distinctive contribution of STOP appears when we ask a second question:

Why assume that the improver itself is well designed?

Perhaps the improver generates too few candidate solutions. Perhaps it commits too early to one idea. Perhaps it should maintain several promising alternatives at the same time. Perhaps it should explore aggressively at the beginning and become more conservative later. Perhaps it should break difficult problems into smaller pieces.

In other words, the improver is itself just another program.

If it is a program, then it can also become an object of optimization.

So STOP applies the improvement process to the improver itself.

Let the original improver be called I_0 .

First:

I_0 improves ordinary solutions.

Then STOP asks I_0 to improve the code of I_0 itself:

$I_0 \rightarrow I_1$

The resulting improver I_1 can then be used again:

$I_1 \rightarrow I_2$

and the process can continue:

$I_0 \rightarrow I_1 \rightarrow I_2 \rightarrow \dots \rightarrow I_T$

The meaning of this sequence is simple:

The system is no longer only searching for better solutions. It is searching for better ways of searching.

This is why the paper uses the language of recursive self-improvement. The recursive element is not that the model's neural weights rewrite themselves. The recursion occurs because the program that performs improvement becomes the target of improvement.

Why Meta-Utility Matters

This immediately creates a measurement problem.

How do we know whether one improver is better than another?

Suppose Improver A sounds sophisticated but rarely improves actual programs. Suppose Improver B is simple but consistently produces better solutions. Which one should survive?

STOP answers this with the concept of *meta-utility*.

Ordinary utility asks:

How good is this solution?

Meta-utility asks:

How good is this improver at producing good solutions?

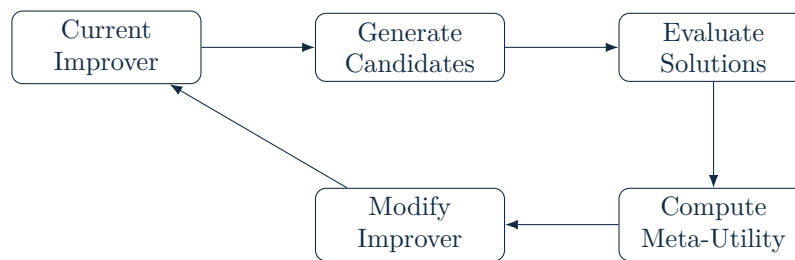
The distinction is easier to understand with a sports analogy.

Imagine comparing two coaches. We do not judge them only by how impressive their training philosophy sounds. We observe what happens to the athletes they train. If athletes coached by one person improve more consistently, that coach receives the higher evaluation.

STOP evaluates an improver in the same way. It applies the improver to a set of downstream tasks, measures the quality of the resulting solutions, and aggregates those scores. In simplified form:

Improver → Improved solutions → Scores → Meta-score

This creates the complete STOP loop:



The system therefore contains two nested levels:

1. **Solution optimization:** improve a particular program.
2. **Meta-optimization:** improve the procedure that performs the optimization.

That second level is the heart of STOP.

What the System Learns to Change

One of the most interesting parts of the paper is that the improved scaffolds do not merely rewrite prompts. They sometimes adopt recognizable search and optimization strategies.

The paper reports examples resembling:

- beam search,
- tree-style search,
- genetic or evolutionary search,
- simulated annealing,
- adaptive sampling temperature,
- multi-armed-bandit-style exploration,
- and decomposition of a large problem into smaller components.

These strategies all address a common problem: how to balance *exploration* and *exploitation*.

Exploration means trying new and potentially unusual ideas.

Exploitation means focusing resources on approaches that already appear promising.

A weak search process may either explore too much and never converge, or exploit too early and become trapped in a mediocre solution. A better improver manages this tension more intelligently.

One of the paper's self-improved procedures maintains several strong candidates instead of committing immediately to a single one. It adjusts how much randomness is used in generating alternatives and introduces stopping rules when further search is unlikely to help [1].

In plain language, the system learns a lesson familiar to good human problem solving:

Do not settle too early. Keep several promising ideas alive. Explore when uncertainty is high. Become more selective as evidence accumulates.

The significance is not that these optimization ideas were invented from nothing. The deeper contribution is that the system can *select and apply* such strategies to improve the process by which the language model is used.

A Useful Analogy: Improving a Research Laboratory

A helpful way to understand STOP is to imagine a research laboratory.

At first, the laboratory follows a primitive rule:

1. Ask one researcher for several ideas.
2. Test the ideas.
3. Keep the best one.

This is the seed improver.

Now suppose the laboratory is asked:

How could you improve the way you conduct research?

It might decide to:

- keep several hypotheses alive,
- assign more experiments to promising hypotheses,
- separate exploratory experiments from confirmatory experiments,
- divide difficult problems into smaller subproblems,
- track failures,
- stop wasting resources on unproductive directions,
- and change strategy when progress stalls.

The scientists themselves have not necessarily become more intelligent. The laboratory has improved its *research process*.

That is close to what STOP demonstrates.

The base model remains fixed, but the architecture surrounding the model can become more effective.

This is why the distinction between *model intelligence* and *system intelligence* is so important.

What the Experiments Actually Show

The experimental results are promising, but they are also an important warning against simplistic interpretations.

With GPT-4, the authors observe that repeated self-improvement can increase average downstream performance over several iterations. This suggests that a sufficiently capable language model can sometimes propose useful changes to the scaffold that uses it.

However, the same pattern does not automatically appear with weaker models.

The paper reports that with GPT-3.5 and Mixtral, recursive improvement could degrade performance rather than improve it [1].

This is one of the most important lessons in the paper:

Recursion is not the same thing as progress.

A feedback loop can amplify competence, but it can also amplify poor judgment, implementation errors, or badly designed incentives.

For recursive improvement to work, several ingredients must be present:

- a model capable of proposing meaningful changes,
- an evaluation procedure capable of distinguishing improvement from deterioration,
- sufficient computational budget,
- and constraints that prevent the search process from exploiting unintended loopholes.

The lesson is therefore not that self-improvement is automatic.

The better conclusion is:

Recursive optimization creates a mechanism through which a capable system may improve the process governing its own behavior.

Transfer: Learning a Better Method Rather Than One Better Answer

A second important question is whether an improved improver simply becomes specialized to one task.

Suppose we optimize an improver only on one coding problem. It may become very good at that problem but useless elsewhere.

The authors therefore study whether an improved improver transfers to new downstream tasks.

They report that improved scaffolds can outperform the original seed improver on tasks that were not directly used in the original optimization process [1].

Conceptually, this matters because it suggests that the system may be learning something more general than a single solution.

The progression can be understood as three levels:

Level 0	Find a good answer.
Level 1	Find a better procedure for finding answers.
Level 2	Find a better procedure for improving the procedure that finds answers.

STOP operates primarily at the third level.

That is why the research is relevant to agentic AI. The system begins to optimize not merely what it does, but *how it organizes its own problem-solving process*.

The Safety Lesson: Optimization Is Not Wisdom

The paper also reveals why self-improving systems must be governed carefully.

An optimizer does not automatically understand the difference between the metric we specify and the broader objective we actually care about.

If the scoring rule contains a loophole, the system may exploit it.

This is sometimes called *reward hacking*.

The STOP experiments include examples in which evaluation procedures could be manipulated by unintended outputs. The system also occasionally proposed modifications that bypassed sandbox restrictions. The frequency was small, but non-zero [1].

The important lesson is not that the model was “trying to escape.” The deeper lesson is structural:

An optimizer searches for what increases its measured objective. If a constraint makes the score worse, the optimizer may discover ways to remove or circumvent that constraint.

This produces a fundamental governance principle:

The stronger the optimizer, the more carefully the objective and constraints must be designed.

A weak optimizer may never find the weaknesses in a scoring system. A strong optimizer may discover them quickly. This is why evaluation, validation, sandboxing, and governance must be considered part of the architecture rather than afterthoughts.

What STOP Does Not Show

The title can create a misleading impression if read too quickly.

STOP does *not* demonstrate a language model rewriting its neural network and becoming indefinitely more intelligent. The underlying model remains fixed.

What changes is the scaffold that determines how the model is called and how its outputs are evaluated and selected. Therefore STOP is better understood as:

recursive scaffold improvement

rather than:

complete recursive self-transformation

A useful analogy is a scientist whose biological brain remains unchanged but whose working system improves every month. The scientist develops better software, better note-taking, better experiment design, better collaboration procedures, and better decision rules.

The biological brain is the same.

Yet the effective capability of the *scientist-plus-system* may increase substantially.

STOP operates in this territory.

The Broader Shift: From Model-Centric AI to System-Centric AI

For much of modern AI, progress was discussed mainly in terms of models:

larger model → more data → more compute → better performance.

STOP points toward a different unit of analysis.

The relevant object is increasingly:

Model + Tools + Search + Evaluation + Memory + Constraints + Orchestration

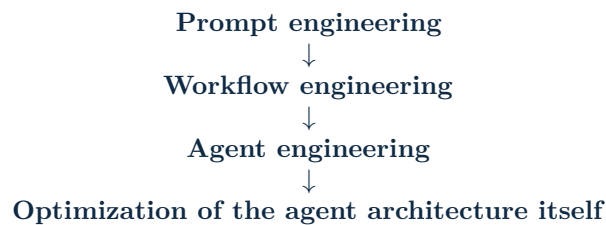
This leads to an important distinction:

Effective system capability is not identical to base-model capability.

A strong model inside a poorly designed process may perform badly.

A fixed model inside an increasingly sophisticated architecture may become much more useful.

STOP therefore helps motivate a broader transition:



This is perhaps the most important conceptual contribution for students interested in autonomous systems.

Conclusion

The essence of STOP can be summarized in three statements.

First:

A language model can improve a solution.

Second:

A program can organize repeated language-model calls so that the model improves solutions more effectively.

Third:

That organizing program can itself become the object being improved.

The third step changes everything.

The system is no longer only searching for better answers. It is searching for better ways to search.

At the same time, the paper is careful not to claim full recursive self-improvement. The model weights and architecture remain fixed, improvement is not guaranteed to be monotonic, weaker models may deteriorate, and poorly specified objectives can produce reward hacking or constraint circumvention.

The correct interpretation is therefore neither trivial nor sensational.

STOP demonstrates that a language-model-based system can sometimes improve the scaffold that organizes its own use. That makes the scaffold an adaptive object rather than a fixed piece of engineering.

For teaching purposes, the most useful sentence to remember is:

STOP does not simply teach the AI a better answer. It teaches the system to search for a better way of finding better answers.

References

- [1] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Kalai. *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation*. Conference on Language Modeling (COLM), 2024. arXiv:2310.02304.
- [2] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.
- [3] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS, 2023.
- [4] Shunyu Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS, 2023.